

Sergey Holubenko ForkLog Article - Grok

Exported: 08/28/2026 02:20

Source: <https://grok.com/>

 USER

<https://forklog.com/exclusive/lyudi-eto-bag> тоже салайн чей ?

 ASSISTANT

Нет, это не «салайн» (и не кто-то с похожим ником/именем).

Статья «Люди — это баг» на ForkLog (опубликована 14.08.2026) написана **Сергеем Голубенко** (авторский профиль: forklog.com/author/Holubenko).

Это большой материал о том, как социальная инженерия, низкая явка в DAO, атаки на governance (пример с BonkDAO и ~\$20 млн) и ИИ-анализ смарт-контрактов ломают идею децентрализации. Автор подробно разбирает человеческий фактор как главную уязвимость, статистику взломов 2025–2026 годов, проблемы аудитов и то, как LLM уже находят эксплойты.

Голубенко — постоянный автор ForkLog, пишет на темы крипты, ИИ, безопасности и смежные. Никакой связи с «Салайн» / Salline / Saline (ни по имени, ни по стилю, ни по предыдущим публикациям) в открытых источниках нет.

Если «салайн» — это конкретный автор/ник из другого материала или канала, который вы имели в виду раньше, уточните ссылку или контекст — тогда можно точнее сравнить. Пока по этой статье ответ однозначный: автор Сергей Голубенко.

Примеры атак на управление DAO

ASSISTANT

Примеры атак на управление DAO (governance attacks) — это случаи, когда злоумышленник использует легитимные механизмы голосования DAO (обычно на основе токенов), чтобы провести вредоносное предложение и вывести средства из казны, захватить контроль или изменить правила. Код смарт-контрактов при этом часто работает «как задумано» — уязвимость лежит в дизайне governance (низкий кворум, отсутствие timelock, пассивность сообщества, мгновенное исполнение предложений).

Такие атаки стали особенно заметны в 2022–2026 годах. Ниже — ключевые реальные примеры, сгруппированные по типам и хронологии, с деталями, нюансами и последствиями.

1. Классические flash-loan атаки (мгновенный захват голосов)

Beanstalk Farms (апрель 2022)

Самый известный и дорогой пример. Атакующий взял flash-loan на >\$1 млрд (Aave, Uniswap, SushiSwap), купил супербольшинство голосов BEAN, провёл предложение BIP-18 (перевод всех средств казны себе) и BIP-19 (для прикрытия — «донат Украине»). Всё произошло в одном блоке: голосование → исполнение → возврат кредита. Убыток ~\$182 млн (прибыль атакующего ~\$76–80 млн).

Почему сработало: отсутствие задержки исполнения и возможность голосовать свежкупленными токенами. После этого многие протоколы ввели защиту от flash-loan voting и timelock.[Spark](#)

Аналогичные (меньшего масштаба) flash-loan атаки случались и позже (например, GreenField DAO в 2025 — ~\$31 млн).

2. Постепенная скупка токенов + низкая явка («apathy attacks»)

BonkDAO (6 июля 2026)

Самый громкий пример 2026 года. Атакующий:

- 30 июня подал BIP #76 («Sowellian BonkDAO») — внешне «реформа и вознаграждение yes-voters», внутри — перевод 4,43 трлн BONK (~\$20 млн) на свой адрес.
- За несколько дней (в основном 4–5 июля) потратил ~\$4,4 млн на покупку BONK на Bybit/Binance (+ частично через DeFi-кредиты), набрав чуть больше 1% эмиссии (кворум).
- Явка — всего 7 кошельков (~2,9% от 18 000+ участников). Атакующий контролировал ~99,9% голосов.
- Предложение исполнилось автоматически (нет timelock).

Код Realms (Solana) отработал идеально. Это классическая «apathy attack»: пассивность сообщества + низкий кворум + мгновенное исполнение. Команда позже отслеживала кошельки

на биржах и пыталась заморозить средства.[Crypto](#)

Compound (июль 2024, Proposal 289)

Группа Golden Boys / whale «Humpry» вывела ~230 тыс. COMP с бирж, делегировала голоса и провела предложение о выделении 499 тыс. COMP (~\$24 млн) в yield-стратегию, которую они контролировали. Голосование прошло с минимальным перевесом (682k vs 633k). Явка была низкой (типично 4–5%).

В отличие от Bonk, атаку удалось частично остановить переговорами: Humpry согласился отозвать предложение в обмен на легитимный продукт staking. Это показало, что даже крупные DeFi-DAO уязвимы к концентрации голосов и апатии.[Thedefiant](#)

Build Finance DAO (февраль 2022)

Атакующий постепенно скупал BUILD-токены через несколько кошельков, провёл предложение, дающее контроль над контрактом токена, затем минтил новые токены и дренировал пулы. Убыток ~\$470–500 тыс. Предложение почти не анонсировали, чтобы избежать контрголосов.

3. Захват через дешёвые/заброшенные токены и низкий supply

Token of Power / TOP (июнь 2026)

Aragon-based DAO с крошечным supply (всего 16 384 токена). Атакующий за ~662 ETH (через Tornado Cash) купил чуть больше 50%, провёл предложение о минтинге 10 млрд новых TOP и свапнул часть в WETH (~\$1,58–1,59 млн). Нет timelock — всё в одной транзакции.[Thedefiant](#)

Term Labs / Term Finance (август 2026)

Атакующий потратил ~\$951 на controlling stake governance-токенов нескольких vaults (в некоторых электорат был почти пустым). Прошёл предложения, отключил задержку, подменил оракулы и вывел ~\$8,5 млн (ETH + USDC). Один из самых «дешёвых» по ROI примеров.[Crypto](#)

Заброшенные DAO (2026):

- Enoki DeFi Club (июль 2026) — захват за ~\$7,19, вывод ~\$51–112 тыс.
- BarnBridge (2026) — ~\$777–860 тыс. через старые approvals.
- StrongBlock (август 2026) — ~\$72 тыс.

4. Социальная инженерия + бэкдор в предложении

Tornado Cash (май 2023)

Атакующий скопировал легитимное старое предложение, добавил `selfdestruct` (позволяло заменить код), прошёл его под видом «идентичного». Получил ~1,2 млн голосов и полный контроль над governance. Позже частично «сбросил» изменения. Это комбинация социальной инженерии и технического трюка.

Synthetify (октябрь 2023) и Indexed Finance (ноябрь 2023) — спам-предложения для тестирования пассивности + скупка токенов в малоактивных DAO.

Общие паттерны и нюансы

- **Главные факторы успеха:**

- Низкая явка (часто <5%).
- Низкий кворум (1% и ниже).
- Отсутствие или слабый timelock (задержка исполнения).
- Возможность голосовать свежкупленными/заёмными токенами.
- Прямой доступ governance к treasury/mint/upgrade-функциям.
- Пассивность или заброшенность сообщества.
- **Масштаб:** только в 2026 году волна governance-takeovers принесла десятки миллионов убытков (BonkDAO — крупнейший). Исторически суммарные потери от таких атак исчисляются сотнями миллионов.
- **Отличия от классических хаков:** здесь нет бага в коде (reentrancy, overflow и т.д.). Атака «легальна» с точки зрения смарт-контракта. Поэтому аудиты CertiK/Code4rena часто не помогают.
- **Защиты, которые помогают:**
- Timelock (48+ часов) + emergency veto/multisig.
- Высокий кворум + snapshot voting (токены должны быть застейканы заранее).
- Delegation + reputation/quadratic voting.
- Мониторинг предложений в реальном времени (Blockaid, Binance Security и др. уже предотвращали атаки).
- Непривязные токены, vesting, anti-flashloan механизмы.

Эти примеры хорошо иллюстрируют тезис из статьи ForkLog «Люди — это баг»: человеческий фактор (апатия, концентрация власти, доверие к «обычным» предложениям) часто оказывается слабее кода. Виталик Бутерин и другие неоднократно критиковали token-weighted DAO именно за такие уязвимости.

Если нужны более глубокие детали по конкретному кейсу (on-chain транзакции, схемы защиты или сравнение с другими векторами), уточните.

нужны более глубокие детали по конкретному кейсу (on-chain транзакции, схемы защиты или сравнение с другими векторами)

ASSISTANT

Глубокий разбор атак через дешёвые / заброшенные токены и низкий supply

Этот класс атак отличается от «классических» governance-takeover (Beanstalk с flash-loan или BonkDAO с покупкой 1% при низкой явке). Здесь ключевой фактор — **крайне низкая рыночная капитализация governance-токена + почти нулевая активность сообщества**. Атакующий покупает контролирующую долю за копейки (иногда за десятки/сотни долларов), проводит предложение и захватывает либо казну, либо опасные права (mint, upgrade, вывод средств через старые approvals). Код работает штатно — уязвимость в экономическом и организационном дизайне.

Ниже — детальный разбор ключевых кейсов 2026 года с on-chain деталями, механикой, последствиями, схемами защиты и сравнением с другими векторами.

1. Token of Power (TOP) — июнь 2026 (~\$1,58–1,59 млн)

Самый «чистый» пример низкого supply.

- **Supply:** всего 16 384 TOP (MiniMeToken + Aragon DAO).
- **Механика:**
 1. Атакующий вывел ~662–664 ETH из Tornado Cash.
 2. Купил чуть больше 50% supply (8 192,000001 TOP) — абсолютное большинство.
 3. В **одной транзакции** создал, проголосовал и исполнил предложение (нет timelock).
 4. TokenManager сминтил 10 млрд новых TOP на контракт атакующего.
 5. Эти токены были свапнуты в Balancer V1 TOP/WETH-пул (50/50), откуда извлекли 944,2 WETH (~\$1,58 млн).
- **On-chain:**
 - Attacker EOA: `0xff8eF7bC455a57e5893232203052Ce0232b39Fa2` (funded via Tornado).
 - Exploit contract: `0x25c68C44A96518294f5B47D758f98309c6729A21`.
 - Ключевая tx: `0x967aa34c69b7775c718545c7f94d92e965eb5fc553c0f27f6f1a9c65c93ac156`.
 - TOP token: `0x0EBD5eC91680d3B0CEDbb1d5BB61851154D3eDb6`.
 - **Почему сработало:** крошечный fixed supply + отсутствие timelock + прямой mint-доступ у governance. Balancer-пул просто стал «выходом» для инфляции.
 - **Последствия:** средства ушли обратно в Tornado Cash. Проект фактически мёртв.

2. Term Finance / Term Labs — 23 августа 2026 (~\$8,5 млн)

Самый крупный и поучительный кейс 2026 года в этом классе.

- **Особенность:** vaults (ERC-4626 на Yearn V3) имели **почти пустой электорат**.
- 5 USDC strategy vaults: никто не wrapping'ал shares → voting power = 0.
- ETH Meta Vault: один старый holder с 0,05 shares (с апреля 2025).
- **Механика:**
 1. Seed: 2 ETH из Tornado Cash.
 2. 17–21 августа: атакующий внёс по ~\$5 в каждый USDC-vault и 0,5 ETH в ETH Meta Vault, сделал wrap → получил 100% голосов в 5 vaults и ~90,66% в ETH Meta.
 3. Подал предложения (сидели 6 дней).
 4. Первое действие после прохождения — **отключение 7-дневного timelock**.
 5. Подмена price oracle на свой контракт → вывод средств.
- **On-chain** (по Defimon / PeckShield / CertiK):
- Attacker: `0xa908b3472d76e7744bab0a5911768a4a6300612b` .
- Ключевая exploit-tx: `0x184f2e57b4ce135181fa2a2166ac394339016338` .
- Консолидация: `0xd5183d8b...fc13` (2 843 ETH + ~1,68 млн DAI).
- Выведено: ~2 843 ETH (~\$6,87 млн) + 1,68 млн USDC (свапнуто в DAI).
- **Почему сработало:** кастомный governance-wrapper вокруг Yearn V3 + пустые vaults + возможность отключить защиту через то же governance. Yearn подчеркнул, что стандартные vaults не затронуты.
- **Последствия:** Term Labs навсегда закрыл Meta Vaults, отозвал DAO-роли. TVL vaults упал на ~68%.

3. Зброшенны́е / dormant DAO

Enoki DeFi Club (31 июля 2026)

Aragon-DAO 2020 года, последний раз трогали в 2020.

- Стоимость захвата: ~\$7,19 (покупка ENOKI + gas + slippage).
- Первое предложение за 6 лет прошло 2149:0.
- ExecuteVote → вывод ~\$51–112 тыс. (ROI ~7000х).
- Кошелёк атакующего был 3-дневный.

BarnBridge SMART Yield (15 июля 2026)

- Стоимость: ~\$600 (32 000 BOND).
- Предложение выглядело как «обычный проху upgrade».
- После прохождения — замена controller на malicious → sweep старых USDC-approvals (~50 кошельков).
- Убыток ~\$776–777 тыс.
- Blockaid предупреждал о рисках approvals заранее.

StrongBlock (6 августа 2026)

- Захват abandoned Governor.
- Предложение: `setPendingAdmin(attacker)` → upgrade на malicious implementation с `forward(address, bytes)`.
- Вывод 32 695 STRONG + 383 447 STRNGR (~\$72 тыс.).

Сравнение с другими векторами

Вектор	Стоимость атаки	Скорость	Зависимость от явки	Типичный ROI	Главная защита
Низкий supply / abandoned	\$7-\$600-\$1-2k	Часы-дни	Почти не нужна	1000-7000x	Минимальный supply + timelock + renounce
Flash-loan (Beanstalk)	Почти 0 (кредит)	1 блок	Не нужна	Высокий	Anti-flashloan + delay
Покупка кворума при низкой явке (BonkDAO)	\$4+ млн	Дни	Критична	4-5x	Высокий кворум + timelock + мониторинг
Социальная инженерия (Tornado Cash 2023)	Средняя	Дни	Нужна	Средний	Code review + Snapshot + multi-sig

Ключевые отличия: в «дешёвых» атаках ROI максимален, потому что цена governance-токена уже близка к нулю, а активы (казны, approvals, vaults) ещё живы. Это «охота на зомби-DAO».

Схемы защиты (что реально работает)

- Timelock обязателен** (минимум 48-168 часов) + невозможность его отключить одним голосованием.
- Минимальный/динамический кворум** + snapshot voting (токены должны быть застейканы заранее).
- Renounce ownership / burn admin-ключей** для dormant-проектов.
- Разделение прав:** governance не должен напрямую контролировать mint/treasury/upgrade.
- Мониторинг в реальном времени** (Blockaid, Defimon, PeckShield) + автоматические алерты.
- Revoke approvals** — пользователи должны периодически проверять старые разрешения (особенно после 2023-2025).
- Quadratic / reputation voting** или non-transferable токены (Dxdao-подход).
- Для vaults: governance только через curator + LP-veto, который нельзя обойти.

Эти атаки 2026 года показали, что даже «мёртвые» протоколы остаются опасными, пока у них есть живые approvals, казна или upgradeable contracts. Самый дешёвый путь к миллионам сейчас — не искать zero-day, а купить голоса в забытом DAO.

Если нужно ещё глубже (конкретные tx-хэши, код предложений, или разбор защиты для конкретного фреймворка вроде Aragon/Realms/Yearn) — уточните кейс.

Разбор транзакций Term Finance

ASSISTANT

Разбор транзакций Term Finance (governance takeover, 23 августа 2026)

Это классический пример атаки через «пустой электорат» + низкий cost of control. Атакующий потратил порядка \$25–\$951 (по разным оценкам) на получение 100% голосов в пяти USDC strategy vaults и ~90–91% в ETH Meta Vault, отключил 7-дневный timelock и вывел ~\$8,46–8,5 млн. Код Yearn V3 + кастомный governance-wrapper Term Labs отработал штатно.

Ключевые адреса

- **Основной атакующий:** `0xa908b3472d76e7744bab0a5911768a4a6300612b`
- **Второй атакующий / helper:** `0x686457a7468b9b31c5dba43b1b16077b48520691`
- **Консолидация средств:** `0xD5183d8BfC65a50863C62aF2538198A8288FFc13`
- **Exploit / malicious strategy contract:** `0x184f2E57b4cE135181FA2A2166AC394339016338`

Хронология и ключевые транзакции

1. Подготовка (17–21 августа 2026)

- 17 августа: кошелёк `0xa908...612b` получает 1 ETH из Tornado Cash (1-ETH pool).
- Второй кошелёк получает ещё 1 ETH из Tornado.
- 17 августа: покупка ~0,485216 shares ETH Meta Vault за ~0,5 ETH → wrap → ~90,66% голосов (до этого был только один старый holder с 0,05 shares).
- 21 августа ~05:26 UTC: executor-контракт `0x4f4b614d...` вносит по ~5 USDC в каждый из пяти USDC strategy vaults и делает wrap. Поскольку никто раньше не wrapping'ал shares, атакующий получает **100% voting power** во всех пяти DAO. Snapshot totalSupply совпадает с балансом атакующего.

2. Подача и голосование предложений

Предложения сидели on-chain 6 дней.

- Единственный голос — атакующего.
- Vetoes: 0.
- Голосование по ETH Meta Vault: block ~25 772 700.

3. Исполнение (23 августа 2026)

Tx #1 — ETH Meta Vault

- Hash: `0xd354a15b15cb73d30908f411aee3f795ec86737a4d080e9a818ac4d6d3014129`
- Время: 06:25:47 UTC (block 25 816 049)
- Voting window закрылось за 12 секунд до этого (06:25:35).
- Что произошло внутри:

1. `executeProposal()`
 2. Первым действием — **обнуление timelock** (`txCooldown()`) с 608 400 секунд → 0).
 3. Добавление / смена strategy на malicious-контракт `0x184f2E57...6338` .
 4. `UpdatedMaxDebtForStrategy` → практически unlimited debt.
 5. Вывод ~2 841,74 WETH (через Aave stataEthWETH wrapper и другие пути).
- Gas: минимальный.

Tx #2 — пять USDC strategy vaults

- Hash: `0x9f273f9a5a20c2fc957b06bbfa45db486390eede4a7f44fbe1a2eb6744c2e8a0`
- Время: ~06:47:47 UTC (примерно через 22 минуты).
- Аналогичная логика: `executeProposal` → отключение задержки → смена oracle/strategy → вывод 1 679 639,29 USDC.

4. Консолидация и обнал

- Перевод ETH: ~2 841,237 ETH ушли на `0xD5183d8B...Fc13` (tx `0xb3971dcb...c479` , block 25 816 079, 06:31:47 UTC).
- USDC → DAI через KyberSwap Meta Aggregation Router (`0x6131b5fa...37b5`): tx `0x92b2aaf0...65ce` (06:48:35 UTC) → 1 679 642,45 DAI.
- Итого на consolidation-адресе (на момент анализа): ~2 843,20 ETH + 1 679 642 DAI.
- Позже часть ETH (300 ETH) ушла в Tornado Cash тремя траншами по 100 ETH через relay.

Технические детали атаки

- **Электорат:** в USDC-vaults был нулевой (никто не wrapping'ал). В ETH Meta — один микро-holder.
- **Защиты, которые обошли:**
- 7-дневный timelock — отключили первым действием внутри того же предложения.
- LP-veto — никто не голосовал (электорат пустой).
- Malicious strategy (`0x184f2E57...`) позволяла произвольные действия после смены.
- Все вызовы шли через `ExecutionFromModuleSuccess` (Yearn/Term module path).

Итоги и реакция

- Потери: ~2 843 ETH (~\$6,87 млн) + 1,68 млн USDC → ~\$8,46–8,5 млн (68–78% TVL vaults).
- Основной lending-протокол Term не затронут.
- Term Labs: навсегда закрыли Meta Vaults, отозвали DAO-роли, withdrawals оставили открытыми.
- Yearn: подчеркнули, что это кастомный wrapper Term, а не стандартные V3-vaults.

Это один из самых «дешёвых» по ROI governance-takeover 2026 года. Полная цепочка on-chain прозрачна: от Tornado → депозиты \$5 → wrap → proposal → execute → drain → consolidation → частичный обнал через Tornado.

Если нужны конкретные internal calls, event logs по какому-то tx или сравнение с другими Yearn-based vaults — скажите.

Анализ смарт-контракта Term Labs

ASSISTANT

Анализ смарт-контрактов Term Labs (в контексте атаки 23 августа 2026)

Term Finance состоит из двух основных слоёв:

1. **Основной протокол** (Term Repo + Auctions) — fixed-rate lending через аукционы. Он **не пострадал**.
2. **Periphery-слой** — **Term Strategy Vaults / Meta Vaults** — именно он стал жертвой. Это ERC-4626 tokenized vaults, построенные на **Yearn V3 Tokenized Strategy** + **кастомный governance wrapper** от Term Labs.

Атака не эксплуатировала баг в коде Yearn V3 или основном протоколе. Она использовала легитимные governance-функции кастомного wrapper'a при практически нулевом электорате.

Архитектура затронутых контрактов

Базовый стек Yearn V3:

- **TokenizedStrategy** — централизованная ERC-4626 логика (депозиты, выводы, accounting, profit locking).
- **BaseStrategy** — прокси, который делегирует вызовы в TokenizedStrategy и хранит storage shares/balances.
- Custom Strategy (Term) — расширяет BaseStrategy логикой работы с Term repo-токенами, аукционами, portfolio constraints (maturity cap, concentration limits, reserve ratio и т.д.).

Кастомный governance-wrapper Term Labs (ключевая точка отказа):

- Разделение ролей:
- **Manager** — операционные действия (аукционы).
- **Governor** — риск-параметры, oracle, controller, emergency-функции, добавление/удаление strategies, **update_debt**, **update_max_debt_for_strategy**.
- **LP token holders** (DAO) — право veto на queued governance-транзакции в течение 7-дневного timelock.
- LP-veto должно было инвалидировать вредоносное предложение до исполнения.
- Timelock (**txCooldown**) — 7 дней (608 400 секунд).

ETH Meta Vault (**tmvETH**, пример адреса из анализов: **0x26fCb50e...**) распределял WETH по нескольким curated sub-strategies (Shorewoods ETH, August Digital ETH, Parity Prime ETH, Parity Core ETH и др.). USDC-стратегии работали аналогично.

Как именно сработали контракты во время атаки

1. **Захват voting power** В USDC strategy vaults электорат был нулевым (никто не wrapping'ал shares). Атакующий внёс ~\$5 в каждый → wrap → 100% голосов. В ETH Meta Vault — один

старый holder (0,05 shares). Атакующий купил ~0,485 shares → ~90–91% голосов.

2. **Предложение** Предложения сидели 6 дней. Описание было стандартным («Veto strategy vault parameter change»). Единственный голос — атакующего. Vetoes = 0.

3. **Исполнение (executeProposal)** Внутри предложения (особенно Tx `0xd354a15b...`):

- Первым действием — **обнуление timelock** (`txCooldown` → 0).
- `update_debt` на все легитимные strategies → 0 (с 100% loss tolerance).
- `add_strategy` malicious-контракта `0x184f2E57b4cE135181FA2A2166AC394339016338` («Fixed Recipient WETH Exit Strategy»).
- `update_max_debt_for_strategy` → `type(uint256).max`.
- `update_debt` на malicious strategy → max.
- Средства (WETH / USDC) ушли в attacker-controlled strategy, который просто форвардил активы дальше.

Malicious strategy удовлетворял минимальному ERC-4626 интерфейсу (принимал asset и минтил shares), поэтому vault принял его без проблем.

Уязвимости дизайна (не баги кода)

Компонент	Проблема	Почему это критично
Governance token / shares	Практически нулевой электорат	\$5–\$25 давали 100% контроль
Timelock	Можно было обнулить внутри того же предложения	7-дневная защита исчезла мгновенно
LP-veto	Требует активных LP	При пустом электорате veto невозможно
Strategy management	Governor мог добавлять arbitrary strategy и ставить unlimited debt	Прямой путь к drain
Role separation	Governor имел слишком широкие права на vault assets	LP не могли перехватить

Yearn официально подтвердил: стандартные Yearn V3 vaults не затронуты. Уязвимость — в **кастомном governance-wrapper** Term Labs.

Аудиты и исходный код

- Основной протокол Term (Repo/Auctions) проходил аудиты Sigma Prime, Runtime Verification, Dedaub (2023–2026).
- Yearn V3 vaults — хорошо аудированы.
- Кастомный governance-слой vaults, судя по доступным данным, не имел такого же уровня публичных аудитов, как core-протокол, или аудиты не покрывали сценарий «пустого

электората + self-execution с обнулением timelock».

Репозитории:

- Основной: `term-finance/term-finance-contracts`
- Vaults: `term-finance/yearn-v3-term-vault-contracts`

Выводы и уроки

- Код Yearn V3 и Term core работал корректно.
- Уязвимость — **экономико-организационная**: дешёвый захват control + слишком мощные права governor + возможность отключить собственные защиты.
- Timelock + LP-veto бесполезны, если электорат пуст и governor может менять параметры защиты.
- После атаки Term Labs навсегда закрыл Meta Vaults и отозвал DAO-роли.

Это один из самых чистых примеров 2026 года, когда governance стал attack surface именно из-за дизайна ролей и отсутствия «живого» сообщества вокруг vaults.

Если нужен более глубокий разбор конкретного контракта (например, malicious strategy `0x184f2E57...`, ETH Meta Vault или governor), event logs или сравнение с другими Yearn-based vaults — уточните.

Анализ атаки на Beefy Finance

ASSISTANT

Анализ атак и инцидентов, связанных с Beefy Finance

По состоянию на август 2026 года **прямой крупной атаки на собственные vaults Beefy Finance** (сравнимой с Term Finance \$8.5M или BonkDAO \$20M) **не было**. Beefy — один из самых зрелых yield-агрегаторов, и большинство серьёзных инцидентов, затрагивавших платформу, были **косвенными** (эксплойты underlying-протоколов). Команда традиционно реагирует быстро через механизм **panic**.

Основные инциденты 2025–2026, затронувшие Beefy

1. Stake DAO Exploit (май 2026) — самый релевантный недавний случай

- Атаковали Stake DAO на Arbitrum (минтинг огромного количества vsdCRV / asdCRV).
- Затронут Beefy vault: **Arbitrum Convex CRV/csdCRV/asdCRV**.
- Beefy сразу поставил vault на паузу и предпринял защитные меры.
- Это классический **cascading risk**: эксплойт underlying-токена/стратегии → воздействие на агрегатор.
- Beefy не потерял контроль над собственными контрактами, а быстро изолировал риск.

2. Contagion от Balancer V2 + Stream Finance (ноябрь 2025)

- Балансер V2 потерял ~\$128 млн из-за rounding-error бага.
- Stream Finance потерял ~\$93 млн (off-chain).
- Каскад затронул Euler, Morpho, Silo, Gearbox, Elixir и **Beefy**.
- Beefy пережил стресс-тест относительно хорошо, но пользователи некоторых vaults почувствовали косвенные потери ликвидности/депеги.
- В статье Beefy «The Scars Of Battle» (декабрь 2025) команда прямо признала, что даже при хорошем риск-менеджменте каскадные эффекты всё равно бьют по пользователям.

3. Другие косвенные инциденты

- **KelpDAO** (апрель 2026) — Beefy использовал **panic** и вывел средства из vaults с exposure к rsETH.
- **USR incident** (март 2026) — прямого exposure не было, но были косвенные позиции в Morpho USDC-рынках, которые тоже паниковали.

Исторический контекст и архитектура безопасности Beefy

Beefy использует стандартную схему:

- **Vault** (ERC-4626-like) + **Strategy** контракты.

- Стратегии часто взаимодействуют с внешними протоколами (Curve, Convex, Uniswap, Aave и т.д.).
- Владение — через **timelock + multi-sig** (обычно 6-часовой timelock на изменения стратегий).
- Есть функция **panic** : keeper может мгновенно вывести средства из underlying farm обратно в strategy-контракт и убрать allowances. Это одна из главных причин, почему Beefy часто успевает защитить пользователей при внешних эксплойтах.

Из аудитов (в т.ч. Cyfrin 2024 и более поздних) известны типичные риски yield-агрегаторов, с которыми Beefy сталкивался/боролся:

- Manipulation of share conversion rate (donation / inflation attack).
- Sandwich-атаки при **setPositionWidth** / **unpause** в concentrated liquidity стратегиях.
- Owner-риски (хотя они сильно ограничены timelock + multi-sig).
- First depositor inflation.
- Риски от underlying-токенов (rebasing, fee-on-transfer и т.д.).

Большинство критических findings в аудитах были исправлены.

Сравнение с атакой на Term Finance

Аспект	Term Finance (авг 2026)	Beefy Finance
Тип атаки	Governance takeover (пустой электорат)	В основном косвенные (underlying)
Стоимость атаки	~\$25-\$951	Обычно высокая (нужен эксплойт underlying)
Главная уязвимость	Кастомный governance wrapper	Зависимость от внешних протоколов
Защитные механизмы	Timelock + LP-veto (не сработали)	Panic + timelock + multi-sig
Реакция команды	Заккрытие Meta Vaults	Быстрый panic + пауза vaults
Прямой drain собственных контрактов	Да	Нет (в 2025–2026)

Выводы

Beefy демонстрирует относительно высокую устойчивость именно благодаря:

- Быстрому **panic** -механизму.
- Timelock + multi-sig на критические функции.
- Опыту реагирования на cascading risks.

Главный оставшийся риск для пользователей Beefy — не прямой хак самих vaults, а **качество underlying-стратегий** и ликвидность внешних протоколов. Именно поэтому команда регулярно паузит vaults при любых подозрительных событиях в экосистеме.

Если вы имели в виду конкретный инцидент (например, Stake DAO vault, какой-то старый хак или конкретный vault на определённой сети) — уточните, и я разберу его глубже (с tx, адресами и механикой).

Анализ уязвимостей протокола Balancer V2

ASSISTANT

Анализ уязвимостей протокола Balancer V2

Balancer V2 — один из самых зрелых AMM в DeFi (запущен в 2021 году). Он прошёл десятки аудитов (OpenZeppelin, Trail of Bits, Certora, ABDK и др.), формальную верификацию и годами обрабатывал миллиарды долларов. Несмотря на это, 3 ноября 2025 года произошёл один из крупнейших эксплойтов в истории DeFi.

Главный инцидент: 3 ноября 2025 (~\$128–130 млн)

Затронуты только Composable Stable Pools (CSPv5 и частично v6) на Balancer V2 и их форки (Beets, BEX и др.).

Balancer V3, Weighted Pools, MetaStable Pools (в большинстве случаев) и другие типы пулов **не пострадали**.

Корневая причина — ошибка направления округления (rounding direction error), существовавшая в коде с 2021 года.

Техническая суть уязвимости

В функции `_upscale()` / `_upscaleArray()` (rate scaling для токенов с rate providers, например osETH, wstETH):

- **Upscaling** всегда округлял **вниз** (`mulDown`).
- **Downscaling** использовал bidirectional rounding (вверх или вниз в зависимости от контекста).

Это нарушало фундаментальный принцип DeFi-математики: **округление всегда должно быть в пользу протокола**.

Особенно критично это проявлялось в пути `EXACT_OUT` / `_swapGivenOut()`:

1. Пользователь указывает точный `amountOut`.
2. Сумма проходит через `_upscale()` → округляется вниз.
3. Округлённое значение используется для расчёта `amountIn` через StableMath (инвариант D).
4. Протокол недополучает `amountIn` (пользователь платит меньше, чем должен).

При нормальных балансах ошибка в 1 wei незаметна. Но когда баланс токена доводится до критически низкого уровня (**8–9 wei**), относительная ошибка становится огромной (до 10–12,5% на операцию).

Механика атаки (трёхфазная)

Атакующий в одной `batchSwap`-транзакции (иногда 65+ микро-свопов) делал:

1. **Подготовка** — выводил ликвидность / свопил BPT → underlying, доводя балансы токенов до 8–9 wei (максимизация ошибки округления).

2. **Манипуляция инвариантом** — серия `EXACT_OUT` свопов в обе стороны. Каждый своп немного занижал инвариант D.

3. **Извлечение** — после сильного снижения D (а значит и цены BPT) атакующий выкупал BPT дёшево и/или выводил активы через internal balances.

Результат: инвариант D мог упасть на 98%, цена BPT обваливалась, а атакующий извлекал реальные активы (WETH, osETH, wstETH и др.).

Атака была **мультичейн** (Ethereum, Arbitrum, Base, Optimism, Polygon, Berachain, Sonic и др.) и произошла за ~30 минут.

Почему аудиты пропустили

- Баг существовал с июля–сентября 2021 (введение rate providers в MetaStable / Composable Stable Pools).
- Trail of Bits ещё в 2021 году находили похожие rounding-issues и рекомендовали fuzzing всех направлений округления.
- Формальная верификация Certora доказывала solvency-свойства, но **не покрывала** adversarial rounding в условиях крайне низких балансов + batch-операций.
- Традиционные аудиты проверяли отдельные операции, а не кумулятивный эффект десятков атомарных свопов при специально созданных граничных условиях.

Другие известные уязвимости Balancer V2

- **2023** — precision vulnerability в Boosted Pools (~\$1–2,1 млн).
- **Token frontrun / internal balance issue** (известна с 2023, не эксплуатировалась вживую до определённого момента) — связанная с gas-оптимизацией и токенами, которых ещё нет on-chain.
- Исторические проблемы с access control в некоторых функциях (в том числе упоминания `manageUserBalance` в ранних анализах, хотя основной вектор 2025 года — именно rounding).

Последствия и уроки

- Потери: ~\$128–130 млн (часть средств позже заморожена/возвращена whitehat'ами и через emergency-действия сетей, например hard fork на Berachain).
- Сильный удар по TVL Balancer и доверию к «проверенным» протоколам.
- Каскадный эффект (в том числе на Beefy и другие агрегаторы, которые паузили связанные продукты).

Ключевые выводы для экосистемы:

1. Rounding direction — критически важный класс уязвимостей, особенно в fixed-point math + rate providers.
2. Аудиты и формальная верификация недостаточны без adversarial fuzzing граничных условий (dust balances + batch operations).

3. Composable-дизайн (rate providers + internal balances + batchSwap) создаёт сложные комбинации, которые сложно полностью покрыть.

4. V3 был спроектирован иначе и не содержал этой уязвимости.

Если нужно более глубокое погружение (конкретные функции StableMath, примеры tx, сравнение с Curve StableSwap или разбор recovery-процесса) — уточните.

Разбор функции _upscale

ASSISTANT

Разбор функции `_upscale` в Balancer V2

Функция `_upscale` — одна из ключевых вспомогательных функций в системе масштабирования (scaling) токенов Balancer V2. Она находится в `ScalingHelpers.sol` и используется почти во всех операциях с пулами (свопы, join/exit, расчёт инварианта).

Исходный код

```
/**
 * @dev Applies `scalingFactor` to `amount`, resulting in a larger or equal value depending
 * on whether it needed
 * scaling or not.
 */
function _upscale(uint256 amount, uint256 scalingFactor) pure returns (uint256) {
    // Upscale rounding wouldn't necessarily always go in the same direction: in a swap for
    // example the balance of
    // token in should be rounded up, and that of token out rounded down.
    // This is the only place where we round in the same direction for all amounts, as the
    // impact of this rounding is expected to be minimal.
    return FixedPoint.mulDown(amount, scalingFactor);
}
```

Аналогичная версия для массивов — `_upscaleArray` — делает то же самое в цикле:

```
function _upscaleArray(uint256[] memory amounts, uint256[] memory scalingFactors) pure {
    uint256 length = amounts.length;
    InputHelpers.ensureInputLengthMatch(length, scalingFactors.length);

    for (uint256 i = 0; i < length; ++i) {
        amounts[i] = FixedPoint.mulDown(amounts[i], scalingFactors[i]);
    }
}
```

Что делает функция

Balancer приводит все токены к единой внутренней точности (обычно 18 decimals), чтобы упростить математику StableSwap / Weighted pools.

- `scalingFactor` = $10^{(18 - decimals)} \times \text{token rate}$ (для обычных токенов rate = 1, для yield-bearing вроде osETH / wstETH rate > 1 и меняется со временем).
- `_upscale` умножает `amount` на `scalingFactor` и делит на `1e18` с округлением вниз (`mulDown`).

`FixedPoint.mulDown(a, b)` = $(a * b) / 1e18$ (целочисленное деление, отбрасывает дробную часть).

Почему это стало уязвимостью

Разработчики сами написали в комментарии, что округление в одну сторону «ожидается минимальным». Это предположение оказалось ложным при определённых условиях.

Проблема проявляется в пути `EXACT_OUT` (`_swapGivenOut`):

```
function _swapGivenOut(...) internal virtual returns (uint256) {
    _upscaleArray(balances, scalingFactors);

    // Вот здесь критическая точка
    swapRequest.amount = _upscale(swapRequest.amount, scalingFactors[indexOut]);

    uint256 amountIn = _onSwapGivenOut(swapRequest, balances, indexIn, indexOut);

    // amountIn округляется вверх (в пользу протокола)
    amountIn = _downscaleUp(amountIn, scalingFactors[indexIn]);

    return _addSwapFeeAmount(amountIn);
}
```

Что происходит:

1. Пользователь указывает точный `amountOut` (сколько хочет получить).
2. Этот `amountOut` проходит через `_upscale` → **округляется вниз**.
3. В StableMath используется уже заниженное значение.
4. Рассчитанный `amountIn` получается меньше, чем должен быть.
5. Протокол отдаёт пользователю полный `amountOut`, но получает меньше `amountIn`.

Когда ошибка становится катастрофической

При нормальных балансах (миллионы токенов) потеря 1 wei — ничто.

Но когда баланс токена доводится до **8-9 wei**, относительная ошибка взлетает до 10-12,5%.

Пример из реальной атаки:

- `amount = 17`
- `scalingFactor ≈ 1.058e18` (rate-токен)
- `17 * scalingFactor / 1e18 ≈ 17.98` → после `mulDown` становится **17**
- Относительная потеря почти 5-6% только на одном вызове.

Атакующий специально доводил балансы до этого «rounding boundary», а затем в одной `batchSwap` делал десятки (иногда 65+) микро-свопов. Каждый своп чуть-чуть занижал инвариант `D`. Цена BPT = `D / totalSupply` падала, и атакующий извлекал реальную ценность.

Почему downscale работает по-другому

У `downscale` есть два варианта:

- `_downscaleDown` → `divDown`
- `_downscaleUp` → `divUp`

То есть направление округления выбирается в зависимости от контекста (в пользу протокола).

У `_upscale` такого выбора нет — всегда `mulDown`.

Это и есть **асимметрия**, которую атакующие использовали.

Итог

`_upscale` — классический пример того, как «безопасное на первый взгляд» округление + комментарий «impact expected to be minimal» превратились в уязвимость на \$128 млн. Баг прожил в коде с 2021 года и пережил множество аудитов, потому что проявлялся только при специально созданных граничных условиях (dust balances + rate providers + batch EXACT_OUT свопы).

Если нужно разобрать соседние функции (`_downscaleUp/Down` , `_onSwapGivenOut` , расчёт инварианта StableMath) или показать конкретные числовые примеры из эксплойта — скажите.

Разбор атаки на инвариант D

 ASSISTANT

$$\text{BPT price} \approx \frac{D}{\text{totalSupply}}$$

Анализ ошибок округления FixedPoint

ASSISTANT

Анализ ошибок округления FixedPoint в контексте Balancer V2

В Solidity нет нативной поддержки дробных чисел. Поэтому DeFi-протоколы используют **fixed-point арифметику** — числа хранятся как целые с подразумеваемой точностью (обычно 18 decimals, где `1e18` = 1.0).

Balancer использует библиотеку `FixedPoint` (на базе OpenZeppelin / собственной реализации). Основные операции:

Основные функции округления

```
// Округление вниз (floor)
function mulDown(uint256 a, uint256 b) internal pure returns (uint256) {
    return (a * b) / ONE;    // ONE = 1e18
}

// Округление вверх (ceiling)
function mulUp(uint256 a, uint256 b) internal pure returns (uint256) {
    uint256 product = a * b;
    return product == 0 ? 0 : ((product - 1) / ONE) + 1;
}

// Аналогично для деления
function divDown(uint256 a, uint256 b) internal pure returns (uint256) {
    return a / b;
}

function divUp(uint256 a, uint256 b) internal pure returns (uint256) {
    return a == 0 ? 0 : ((a - 1) / b) + 1;
}
```

Ключевой принцип безопасности в DeFi

Округление всегда должно быть в пользу протокола (или, по крайней мере, не в пользу атакующего).

Операция	Правильное направление	Почему
Пользователь платит (amountIn)	Округлять вверх (mulUp / divUp)	Протокол получает не меньше, чем нужно
Протокол отдаёт (amountOut)	Округлять вниз (mulDown / divDown)	Протокол отдаёт не больше, чем рассчитано
Балансы резервов	Обычно вверх	Пул не выглядит беднее, чем есть
Расчёт инварианта D	Зависит от контекста	Ошибки не должны систематически занижать D

Где именно сломалось в Balancer V2

В `_upscale` :

```
function _upscale(uint256 amount, uint256 scalingFactor) pure returns (uint256) {
  // Комментарий разработчиков:
  // "the impact of this rounding is expected to be minimal"
  return FixedPoint.mulDown(amount, scalingFactor);
}
```

Проблемы:

- 1. **Всегда** `mulDown` , независимо от контекста (вход или выход).
- 2. Когда `scalingFactor` включает **rate** (osETH, wstETH и т.д.), он становится нецелым → truncation становится значимым.
- 3. При очень маленьких `amount` (8-17 wei) относительная ошибка взлетает.

Пример реальной потери точности:

```
amount          = 17
scalingFactor = 1_058_132_408_689_971_699    // ~1.058e18

17 * scalingFactor = 17_988_251_947_729_518_883
разделить на 1e18 → 17.988...

mulDown → 17
```

Абсолютная ошибка ≈ 0.988

Относительная ошибка ≈ **5.8 %**

При балансе 8 wei ошибка в 1 wei = **12.5 %**.

Почему ошибка накапливается

В `batchSwap` атакующий выполнял десятки `EXACT_OUT` свопов подряд:

1. `_upscale(amountOut)` → занижение
2. `StableMath` считает `amountIn` от заниженного значения → ещё одно занижение
3. Балансы обновляются
4. Следующий своп работает уже с искажёнными балансами
5. Инвариант D пересчитывается на искажённых данных → D падает

Каждая операция вносит микро-ошибку, но из-за итеративной природы Newton-Raphson и повторного использования результатов ошибка **компаундируется**.

Сравнение правильного и неправильного подхода

Правильно (в пользу протокола):

- При `upscale amountOut` (то, что протокол отдаёт) → `mulUp`
- При `upscale amountIn` (то, что протокол получает) → `mulDown` или `mulUp` в зависимости от точной семантики

Как было в Balancer:

- `_upscale` всегда `mulDown` → в пути `EXACT_OUT` протокол систематически недополучал.

Уроки по FixedPoint-арифметике

1. **Никогда не делайте однонаправленное округление «на все случаи жизни»**, если операция может использоваться в разных направлениях потока ценности.
 2. **Комментарий «impact expected to be minimal»** — красный флаг. Нужно проверять граничные случаи (dust amounts).
 3. **Rate providers** превращают `scalingFactor` в нецелое число и резко увеличивают опасность truncation.
 4. **Fuzzing + property-based testing** должны специально проверять:
 - очень маленькие balances (1–100 wei)
 - комбинации с `rate ≠ 1`
 - длинные batch-операции
1. Формальная верификация часто доказывает «нормальные» свойства, но пропускает adversarial dust-сценарии.

Итог

Ошибки округления FixedPoint сами по себе почти всегда выглядят безобидно (потеря 1 wei). Но когда они:

- направлены **против** протокола,
- усиливаются rate-scaling,

- повторяются десятки раз в атомарной транзакции,
- влияют на инвариант, от которого зависит цена LP-токена,

— они превращаются в эксплойт на сотни миллионов долларов.

Именно это и произошло с `_upscale` + `mulDown` в Balancer V2 3 ноября 2025 года.

Если нужно разобрать реализацию `FixedPoint` подробнее, сравнить с OpenZeppelin, Uniswap V3 или Curve, или показать математику накопления ошибки — скажите.

Разбор функции `_downscaleUp`

ASSISTANT

Разбор функции `_downscaleUp` в Balancer V2

`_downscaleUp` — это парная функция к `_upscale`. Она переводит значение из внутренней (18-decimal) точности обратно в нативную точность токена с **округлением вверх**.

Исходный код

```
/**
 * @dev Reverses the `scalingFactor` applied to `amount`, resulting in a smaller or equal
 * value depending on
 * whether it needed scaling or not.
 * The result is rounded up.
 */
function _downscaleUp(uint256 amount, uint256 scalingFactor) pure returns (uint256) {
    return FixedPoint.divUp(amount, scalingFactor);
}
```

Где `FixedPoint.divUp` реализован примерно так:

```
function divUp(uint256 a, uint256 b) internal pure returns (uint256) {
    if (a == 0) {
        return 0;
    } else {
        return ((a - 1) / b) + 1; // ceiling division
    }
}
```

Существует также массивная версия `_downscaleUpArray`.

Зачем нужна эта функция

Balancer работает во внутренней точности 18 decimals. После всех математических расчётов (инвариант, свопы и т.д.) результат нужно вернуть в реальные decimals токена.

- `_downscaleDown` → округление **вниз** (`divDown`)
- `_downscaleUp` → округление **вверх** (`divUp`)

Выбор направления зависит от того, в чью пользу должно быть округление.

Когда используется `_downscaleUp`

Главный случай — когда протокол **получает** токены от пользователя (`amountIn`).

Пример из `_swapGivenOut` :

```
function _swapGivenOut(...) internal virtual returns (uint256) {
    _upscaleArray(balances, scalingFactors);
    swapRequest.amount = _upscale(swapRequest.amount, scalingFactors[indexOut]); // ←
```

здесь была проблема

```
uint256 amountIn = _onSwapGivenOut(...);

// amountIn tokens are entering the Pool, so we round up.
amountIn = _downscaleUp(amountIn, scalingFactors[indexIn]); // ← правильно

return _addSwapFeeAmount(amountIn);
}
```

Логика:

- Пользователь должен заплатить `amountIn`.
- Чтобы протокол **не недополучил**, результат округляется **вверх**.
- Пользователь платит чуть больше или ровно столько, сколько нужно (но не меньше).

Аналогично `_downscaleUp` применяется при join'ах (когда пользователь вносит токены в пул).

Сравнение с `_downscaleDown`

Функция	Округление	Когда используется	Цель
<code>_downscaleUp</code>	Вверх	<code>amountIn</code> (протоколу платят)	Протокол не недополучает
<code>_downscaleDown</code>	Вниз	<code>amountOut</code> (протокол отдаёт)	Протокол не отдаёт лишнего

Это **правильное** bidirectional-поведение.

Почему `_downscaleUp` не спасла от эксплойта

Проблема была **не** в `downscale`, а в `upscale`.

В пути `EXACT_OUT`:

1. `amountOut` (то, что протокол **отдаёт**) проходил через `_upscale` → **mulDown** (занижение).
2. `StableMath` считала `amountIn` уже от заниженного `amountOut`.
3. Потом `_downscaleUp` честно округляла этот (уже заниженный) `amountIn` вверх.
4. Но «вверх от заниженного числа» всё равно давало меньше, чем должно было быть при правильном `upscale`.

То есть `_downscaleUp` работала корректно, но получала на вход уже «отравленное» значение.

Важный принцип

Правильная пара должна выглядеть так:

- При `upscale` `amountOut` (обязательство протокола) → желательно **mulUp**
- При `downscale` `amountIn` (требование к пользователю) → **divUp** (`_downscaleUp`)

В Balancer V2 `upscale` был однонаправленным (`mulDown` всегда), а `downscale` — двунаправленным. Эта асимметрия и стала одной из ключевых причин уязвимости.

Итог

`_downscaleUp` — пример **хорошей** практики округления в пользу протокола. Она использовалась правильно. Уязвимость возникла из-за того, что парная операция `_upscale` не имела аналогичного выбора направления и всегда округляла вниз, в том числе в ситуациях, когда нужно было округлять вверх.

Если хотите, могу разобрать `_downscaleDown` или показать полный путь данных `amountIn/amountOut` с конкретными числами.

Разбор ошибки эксплойта

ASSISTANT

Разбор ошибки эксплойта Balancer V2 (ноябрь 2025)

Корневая ошибка одной фразой

В функции `_upscale` всегда использовалось округление **вниз** (`mulDown`), даже когда значение представляло обязательство протокола (`amountOut` в `EXACT_OUT` свопах). Это нарушало базовый принцип: **округление должно быть в пользу протокола**.

Где именно была ошибка

```
// ScalingHelpers.sol
function _upscale(uint256 amount, uint256 scalingFactor) pure returns (uint256) {
    // Комментарий разработчиков:
    // "the impact of this rounding is expected to be minimal"
    return FixedPoint.mulDown(amount, scalingFactor); // ← всегда вниз
}
```

В пути `_swapGivenOut` (Exact Output):

```
swapRequest.amount = _upscale(swapRequest.amount, scalingFactors[indexOut]);
// amountOut округляется ВНИЗ

uint256 amountIn = _onSwapGivenOut(...); // считается от заниженного amountOut

amountIn = _downscaleUp(amountIn, scalingFactors[indexIn]); // честно вверх, но уже поздно
```

Почему это критично

1. Пользователь указывает `amountOut` — сколько токенов он хочет получить.
2. Протокол должен рассчитать, сколько `amountIn` пользователь обязан заплатить.
3. Из-за `mulDown` `amountOut` занижается → Math считает, что нужно меньше `amountIn`.
4. Протокол всё равно отдаёт полный (оригинальный) `amountOut`.
5. Результат: протокол **недополучает** оплату.

При обычных балансах ошибка = 1 wei (ничтожно).

При балансах **8–9 wei** + rate-токенах (osETH, wstETH) относительная ошибка достигает 5–12 %.

Как ошибка превратилась в эксплойт

Атакующий делал три вещи:

1. **Доводил баланс** одного токена до 8–9 wei (максимизация относительной ошибки).
2. **Повторял EXACT_OUT свопы** десятки раз в одном `batchSwap` (ошибка накапливалась).

3. **Ломал инвариант D** — каждый своп немного занижал D → цена BPT падала → можно было извлекать реальную ценность.

D мог упасть на 98 %. После этого атакующий выводил активы.

Почему ошибка прожила 4 года

- Появилась в 2021 году вместе с rate providers в Composable Stable Pools.
- Разработчики сознательно выбрали однонаправленное округление, написав «impact expected to be minimal».
- Аудиты и формальная верификация проверяли «нормальные» сценарии, но почти не проверяли adversarial dust-balances + длинные batch-операции.
- `_downscale` имел правильное bidirectional-поведение (`Up` / `Down`), а `_upscale` — нет. Асимметрия осталась незамеченной.

Суть ошибки в терминах безопасности

Что должно быть	Что было фактически
Округление в пользу протокола	Округление против протокола
Разные направления в зависимости от потока ценности	Всегда mulDown
Защита от dust-amounts	Максимальная уязвимость именно на dust

Итог

Ошибка — не в сложной математике StableSwap и не в отсутствии access control.

Это **неправильный выбор направления округления** в одной маленькой helper-функции + ложное предположение, что «1 wei ни на что не влияет».

Именно эта микро-ошибка, усиленная rate-scaling и batch-операциями, позволила за ~30 минут вывести ~\$128 млн с нескольких сетей.

Если нужно, могу показать полный поток данных на числовом примере или сравнить, как эту же проблему решили в Balancer V3.